

Word2Vec para análisis de sentimientos en español: arquitectura, optimización y comparación de algoritmos de entrenamiento

Danna Nickol Gala Vásquez Christopher Renato Perez Torres
Daniel Davis Villalobos Carlos Paul Ricardo Maguiña Quispe
Oswaldo Alejandro Quispe Monzón

Curso: Optimización (DS2011) — Docente: Victor Antonio Torrez Castillo
Junio 2026 — Lima, Perú

Resumen

La idea de este trabajo, en una línea, es enseñarle a una computadora a leer una reseña y decidir si es positiva o negativa — y mirar todo ese proceso con los lentes de la **optimización**. Para lograrlo estudiamos **Word2Vec**, un modelo que convierte palabras en números, y lo implementamos desde cero en PyTorch (no usamos una librería que lo haga por nosotros). Comparamos dos arquitecturas (una y dos capas) y tres algoritmos para entrenarlo (SGD con momentum, RMSProp y Adam), cada uno con su tasa de aprendizaje adecuada. El estudio se arma en seis experimentos y una verificación del gradiente: comparar arquitecturas y optimizadores; medir qué tan sensible es SGD a la tasa de aprendizaje; rastrear la norma del gradiente para entender *por qué* unos algoritmos avanzan y otros no; comparar contra el método clásico TF-IDF; revisar la semántica que aprende el modelo; buscar los mejores hiperparámetros; y, al final, un *duelo* de nuestro modelo contra unos embeddings preentrenados con miles de millones de palabras (SBWC), dentro y fuera de su dominio.

Lo más importante que encontramos: (i) con su tasa adecuada, SGD sí funciona (F1-macro = 0,861), pero es *muy* sensible a ese ajuste; a la tasa de Adam su gradiente es unas 80 veces más pequeño y se queda atascado; (ii) el mejor Word2Vec llega a F1-macro $\approx 0,89-0,90$; (iii) el clásico TF-IDF nos gana, incluso en su versión más simple (0,911), lo que dice más del problema que de nuestro modelo; (iv) la búsqueda Bayesiana no mejora a una rejilla bien diseñada; y (v) en el duelo final, *nuestro modelo le gana al gigante preentrenado en nuestro propio dominio* (0,903 vs. 0,883), pero el gigante generaliza mejor fuera de él, y la razón cabe en un solo número: la cobertura de vocabulario (82,7 % vs. 96,4 %). Para cerrar el círculo entre teoría y práctica, verificamos numéricamente que el gradiente que derivamos a mano es correcto.

Palabras clave: Word2Vec, Skip-gram, negative sampling, descenso del gradiente, RMS-Prop, Adam, Optuna, transferibilidad, NLP en español.

Índice

1. Introducción	3
1.1. El problema, en simple	3
1.2. ¿Qué tiene que ver con Optimización?	3
1.3. Cómo está organizado el documento	3
2. Planteamiento del problema	3
2.1. Pregunta de investigación	3
2.2. Hipótesis de trabajo	3

3. Marco teórico	4
3.1. Del texto a los vectores: el problema de fondo	4
3.2. La idea central de Word2Vec	4
3.3. Arquitectura Skip-gram	4
3.4. Negative Sampling: nuestra función de pérdida	4
3.5. El gradiente: hacia dónde empujar	5
3.6. Un punto fino: este problema NO es convexo	5
3.7. La extensión de dos capas	5
3.8. Los tres optimizadores	5
3.9. Del vector de palabra al clasificador	6
4. Metodología	6
4.1. El dataset	6
4.2. Preprocesamiento	6
4.3. Configuración y una decisión importante	7
5. Resultados	7
5.1. Curvas de convergencia: ¿bajan el error?	7
5.2. Resultados de clasificación	8
5.3. Visualización: ¿el mapa de palabras existe?	8
5.4. Experimento 1: ¿qué tan quisquilloso es SGD con la tasa?	9
5.5. Experimento 2: ¿por qué Adam avanza y SGD no?	9
5.6. Experimento 3: el clásico TF-IDF entra al ring	10
5.7. Experimento 4: ¿el modelo entendió de verdad?	11
5.8. Experimentos 5 y 5b: buscando los mejores ajustes	12
5.9. Experimento 6: el duelo contra el gigante	13
5.10. Verificación del gradiente	14
6. Análisis y discusión	14
7. Fortalezas, limitaciones y usos profesionales	15
7.1. Fortalezas	15
7.2. Limitaciones	15
7.3. Usos actuales en el ámbito profesional	15
8. Conclusiones	15

1. Introducción

1.1. El problema, en simple

Empecemos por el problema. Cuando alguien deja una reseña y escribe “llegó rápido y funciona genial”, cualquier persona entiende de inmediato que está satisfecha. Una computadora, en cambio, solo ve una secuencia de símbolos sin significado. En el fondo, todo este proyecto consiste en enseñarle a la máquina a dar ese salto: de letras a sentimiento.

1.2. ¿Qué tiene que ver con Optimización?

Una relación muy directa. Cada decisión que tomamos para que el modelo aprenda se reduce a dos cosas: definir una **función de error** —un número que mide qué tan equivocado está el modelo— y buscar, paso a paso, cómo reducir ese número. Ese descenso gradual del error es, precisamente, optimizar. Dicho de otro modo, debajo de un problema de lenguaje se esconde un problema de optimización pura y dura, y eso es lo que iremos destapando.

Los temas del curso que aparecen explícitamente son: los *métodos gradientes* (SGD como caso base), la *regresión logística* (el clasificador final y la forma de nuestra función de pérdida), los *métodos cuasi-Newton* (RMSProp y Adam, que adaptan el paso usando información parecida a la de segundo orden) y las *redes neuronales con backpropagation* (Word2Vec es una red poco profunda). Como valor agregado, incluimos una *verificación numérica del gradiente*: derivamos la fórmula a mano y comprobamos que está bien.

1.3. Cómo está organizado el documento

Seguimos el flujo natural de un experimento: primero la teoría (Sección 3), luego cómo lo montamos (Sección 4), después qué salió (Sección 5), el análisis de esos resultados (Sección 6) y las conclusiones (Sección 8).

2. Planteamiento del problema

2.1. Pregunta de investigación

¿Cómo afecta la elección del algoritmo de optimización (SGD, RMSProp, Adam) y la profundidad de la red (1 vs. 2 capas) al entrenamiento de Word2Vec y a la calidad de los vectores que produce? Y una vez que tenemos nuestro mejor modelo, ¿cómo se compara contra embeddings preentrenados en un corpus gigantesco?

2.2. Hipótesis de trabajo

Antes de correr nada, escribimos lo que esperábamos que pasara. Esto importa: varias de estas hipótesis terminaron *refutadas*, y reportarlo honestamente es parte del trabajo.

- H1.** Los métodos adaptativos (RMSProp, Adam) convergerán más rápido y a menor error que SGD usando la misma tasa de aprendizaje.
- H2.** La arquitectura de 2 capas producirá mejores vectores y, por tanto, mejor clasificación que la de 1 capa.
- H3.** SGD es más sensible a la tasa de aprendizaje que Adam: solo funciona en un rango estrecho, mientras Adam aguanta variaciones de varios órdenes de magnitud.
- H4.** Word2Vec superará a TF-IDF porque captura relaciones de significado que el método clásico no puede ver.

H5. La búsqueda Bayesiana (Optuna) encontrará una configuración bastante mejor que una búsqueda en rejilla.

H6. Los embeddings preentrenados en un corpus enorme (SBWC) generalizarán mejor *fuera de dominio* que los nuestros, entrenados con apenas 40 000 reseñas.

3. Marco teórico

3.1. Del texto a los vectores: el problema de fondo

La forma más simple de darle texto a una computadora es la *bag-of-words*: un vector gigante donde cada posición cuenta cuántas veces aparece una palabra. Tiene dos problemas. El primero es de tamaño: con decenas de miles de palabras, esos vectores son enormes y casi todos ceros. El segundo es más sutil pero más grave: no hay noción de significado. Para ese método, “bueno” y “excelente” son tan distintos entre sí como “bueno” y “mesa”, porque cada palabra vive en su propia posición sin relación con las demás. Word2Vec ataca justamente esos dos problemas.

3.2. La idea central de Word2Vec

Antes de la fórmula, la idea es simple. Para que la máquina trabaje con palabras, primero hay que convertirlas en números. Word2Vec lo hace con una idea elegante: ubica cada palabra como un punto en un espacio, de modo que las palabras de significado parecido queden cerca y las distintas, lejos. Es como un mapa, pero de palabras en lugar de ciudades: “excelente” y “perfecto” quedarían en la misma vecindad; “excelente” y “horrible”, en extremos opuestos.

¿Y cómo decide la máquina quién está cerca de quién, si nadie le explicó el significado de cada palabra? A partir del contexto: si dos palabras suelen aparecer rodeadas de las mismas palabras, es muy probable que signifiquen algo parecido. Esta intuición tiene nombre propio —la **hipótesis distribucional** [1]— y es el corazón de Word2Vec [4].

3.3. Arquitectura Skip-gram

Word2Vec tiene dos sabores; nosotros usamos **Skip-gram**, que funciona así: toma una palabra y trata de adivinar qué palabras la rodean. Entrenando millones de veces esa tarea de “adivina el vecindario”, los vectores se acomodan solos para que las palabras que comparten contexto queden cercanas. Formalmente, para un corpus $w_1, \dots, w_T$ y una ventana de tamaño c , busca maximizar la probabilidad de los vecinos:

$$\mathcal{L}_{SG} = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t). \quad (1)$$

Esa probabilidad se calcula, en teoría, con un *softmax* que compara la palabra contra *todo* el vocabulario. El problema: con miles de palabras, hacer esa cuenta en cada paso es carísimo. La solución se llama *negative sampling*.

3.4. Negative Sampling: nuestra función de pérdida

La intuición del truco es bonita. En vez de preguntarle al modelo “¿cuál de las 9 597 palabras del vocabulario es el vecino correcto?” (carísimo), le hacemos una pregunta mucho más barata: “aquí tienes el vecino verdadero y unas pocas palabras al azar; ¿sabes distinguir al verdadero de los impostores?”. Eso convierte el problema en una simple clasificación binaria. Para un par positivo (w_I, w_O) con K negativos $\{w_k\}$, la pérdida es [5]:

$$\mathcal{L}_{NS} = -\log \sigma(\mathbf{v}_{w_O}^\top \mathbf{u}_{w_I}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{w_k}^\top \mathbf{u}_{w_I}), \quad (2)$$

donde $\sigma(x) = 1/(1 + e^{-x})$ es la sigmoide y los negativos se sacan de una distribución $P_n(w) \propto f(w)^{3/4}$. Detalle que conecta con el curso: la Ecuación (2) es exactamente una **entropía cruzada binaria**, la misma de la regresión logística. Minimizarla respecto a los vectores $\mathbf{u}$ y $\mathbf{v}$ es el problema de optimización que resolvemos.

3.5. El gradiente: hacia dónde empujar

Para minimizar la pérdida necesitamos su gradiente, que nos dice en qué dirección mover cada vector. El del vector de entrada $\mathbf{u}_{w_I}$ es:

$$\frac{\partial \mathcal{L}_{NS}}{\partial \mathbf{u}_{w_I}} = \left[\sigma(\mathbf{v}_{w_O}^\top \mathbf{u}_{w_I}) - 1 \right] \mathbf{v}_{w_O} + \sum_{k=1}^K \sigma(\mathbf{v}_{w_k}^\top \mathbf{u}_{w_I}) \mathbf{v}_{w_k}. \quad (3)$$

Leída en palabras, esta fórmula hace algo muy intuitivo: *acerca* la palabra central a su contexto real y la *aleja* de las palabras al azar. Repetido millones de veces, ese empujón es el que va formando el “mapa de palabras”. Más adelante (Sección 5.10) comprobamos numéricamente que esta fórmula está bien derivada.

3.6. Un punto fino: este problema NO es convexo

Vale la pena aclarar algo que suele malinterpretarse. Un problema convexo es como un tazón: tiene un único fondo y siempre llegas a él. Uno no convexo es como una cordillera llena de valles: dependiendo de por dónde bajes, puedes quedar atrapado en un valle que no es el más profundo. **Nuestra pérdida es no convexa, incluso con una sola capa.** La razón es que en $\sigma(\mathbf{v}^\top \mathbf{u})$ aparecen multiplicados los dos vectores que estamos ajustando, y esa multiplicación entre incógnitas (bilinealidad) crea ese paisaje accidentado. La extensión a dos capas con $\tanh$ *agrega* más relieve, pero el terreno ya era irregular de origen. Esto es justo lo que hace interesante comparar optimizadores: el resultado depende de cómo cada algoritmo navega esa cordillera.

3.7. La extensión de dos capas

Probamos también una versión con una capa extra, que mete una transformación no lineal entre el vector de entrada y la comparación final:

$$\mathbf{h} = \tanh(W_2 \mathbf{u}_{w_I} + \mathbf{b}) \in \mathbb{R}^{d_h}, \quad (4)$$

$$\mathcal{L}_{2L} = -\log \sigma(\mathbf{v}_{w_O}^\top \mathbf{h}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{w_k}^\top \mathbf{h}). \quad (5)$$

La idea era que esa capa extra aprendiera representaciones más ricas. Si funcionó o no, lo vemos en los resultados.

3.8. Los tres optimizadores

Aquí está el corazón del curso. Entrenar es como bajar una montaña con niebla: no ves el fondo, solo sientes la pendiente bajo tus pies (el gradiente) y decides hacia dónde dar el siguiente paso. Lo que cambia entre algoritmos es *cómo* eligen ese paso.

SGD con momentum. El descenso del gradiente estocástico da un paso en contra de la pendiente. Le agregamos *momentum*: como una pelota que rueda cuesta abajo, acumula inercia y no se detiene ante cada pequeña irregularidad.

$$\mathbf{m}_{t+1} = \mu \mathbf{m}_t + \nabla_{\theta} \mathcal{L}(\theta_t), \quad \theta_{t+1} = \theta_t - \eta \mathbf{m}_{t+1}, \quad \mu = 0.9. \quad (6)$$

El momentum no es un lujo aquí: como la matriz de salida arranca en ceros, el primer gradiente es diminuto; sin esa inercia acumulada, SGD ni siquiera se mueve del punto de partida. Su gran debilidad es el tamaño del paso η (la *tasa de aprendizaje*): muy grande y se pasa de largo; muy pequeño y tarda una eternidad.

RMSPProp. Su idea: usar un paso distinto para cada dirección, según qué tan empujada ha venido siendo. Donde el terreno cambia brusco, pasos cortos; donde es suave, pasos largos.

$$v_t = \alpha v_{t-1} + (1 - \alpha)g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t, \quad \alpha = 0,99. \quad (7)$$

Adam. Combina lo mejor de ambos: la inercia del momentum y el paso adaptativo de RMS-Prop, con una corrección al inicio para no arrancar sesgado.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2, \quad (8)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t, \quad (9)$$

con $\beta_1 = 0,9$ y $\beta_2 = 0,999$. Por eso a Adam se le suele llamar “el optimizador que casi siempre funciona sin que lo toques”.

3.9. Del vector de palabra al clasificador

Word2Vec nos da un vector por palabra, pero una reseña tiene muchas palabras. La receta simple: el vector de un documento es el *promedio* de los vectores de sus palabras, $\mathbf{d} = \frac{1}{|T_{doc}|} \sum_w \mathbf{u}_w$. Sobre ese promedio entrenamos una **Regresión Logística** que decide positivo o negativo. (Promediar tiene un costo: se pierde el orden de las palabras, algo que retomamos en las limitaciones.)

4. Metodología

4.1. El dataset

Usamos `amazon_reviews_multi` [2] en español (vía MTEB [6]): reseñas con calificación de 1 a 5 estrellas. Lo volvimos un problema binario: 1–2 estrellas es negativo, 4–5 es positivo, y las de 3 (ambiguas) las descartamos. Entrenamos sobre un subconjunto balanceado de 40 000 reseñas (Cuadro 1). Elegimos este corpus porque es grande, está escrito en español nativo y las estrellas son una etiqueta de sentimiento confiable (las pone el propio comprador).

Tabla 1: Particiones del dataset tras volverlo binario.

Partición	Total	Negativo	Positivo
Entrenamiento	40 000	19 838	20 162
Validación	4 000	≈2 000	≈2 000
Test	4 000	≈2 000	≈2 000

4.2. Preprocesamiento

Lo dejamos a propósito mínimo, para no borrar señal útil: pasar a minúsculas, quitar URLs y menciones, eliminar puntuación y dígitos, separar por espacios y descartar tokens de una sola letra. Con `min_count=5` (ignorar palabras que aparecen menos de 5 veces), el vocabulario quedó en **9 597 palabras**.

4.3. Configuración y una decisión importante

El Cuadro 2 lista los hiperparámetros base. La lógica: fijamos $d = 100$ y todo lo demás para comparar limpio el efecto del optimizador y la arquitectura; después exploramos la dimensión y la tasa por separado. Todas las corridas fijan una semilla aleatoria ($= 42$) para que los resultados sean reproducibles.

Tabla 2: Hiperparámetros base del experimento principal.

Hiperparámetro	Valor	Qué es
Dimensión de embeddings d	100	Tamaño del vector de cada palabra (control fijo)
Dimensión capa oculta d_h	64	Solo en la versión de 2 capas
Ventana de contexto c	5	Cuántas palabras a cada lado se consideran vecinas
Negativos por par K	5	Cuántos “impostores” por cada vecino real
Épocas	5	Pasadas completas por el corpus (7 en exp1/exp2)
Tamaño de batch	512	Pares por actualización

Una tasa de aprendizaje por optimizador. Esto nos lo cuestionamos bastante. Comparar los tres optimizadores con la *misma* tasa parece justo, pero no lo es: 0,001 es la tasa cómoda de Adam y RMSProp, pero a SGD lo deja paralizado. Sería como hacer correr los 100 metros a tres atletas con la misma talla de zapatillas. Así que le dimos a cada uno su tasa adecuada: **SGD con $\eta = 0,1$, RMSProp y Adam con $\eta = 0,001$** . La sensibilidad de SGD a ese ajuste la estudiamos aparte (exp1), que es donde realmente brilla esa observación.

5. Resultados

5.1. Curvas de convergencia: ¿bajan el error?

Lo primero es mirar si el entrenamiento de verdad reduce la pérdida época tras época (Figuras 1 y 2). Con su tasa adecuada, los tres optimizadores bajan; SGD parte más arriba y desciende más lento que los adaptativos, pero ya no se queda clavado como veremos que pasa cuando le ponemos una tasa equivocada.

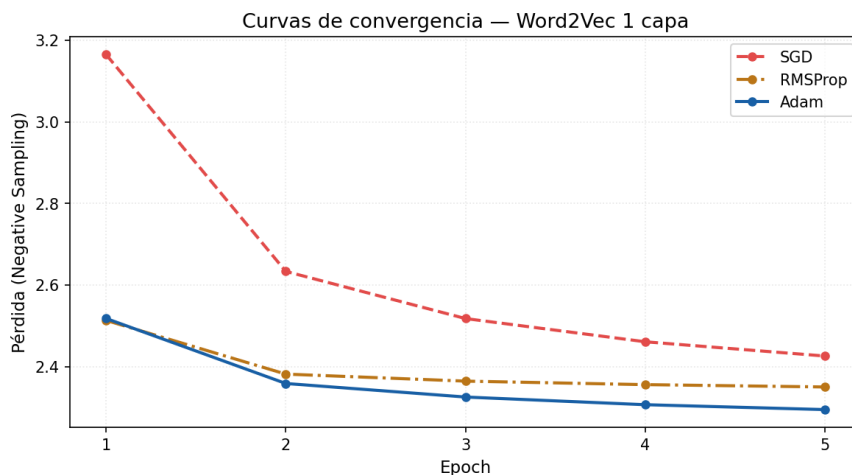


Figura 1: Curvas de convergencia — Word2Vec de 1 capa.

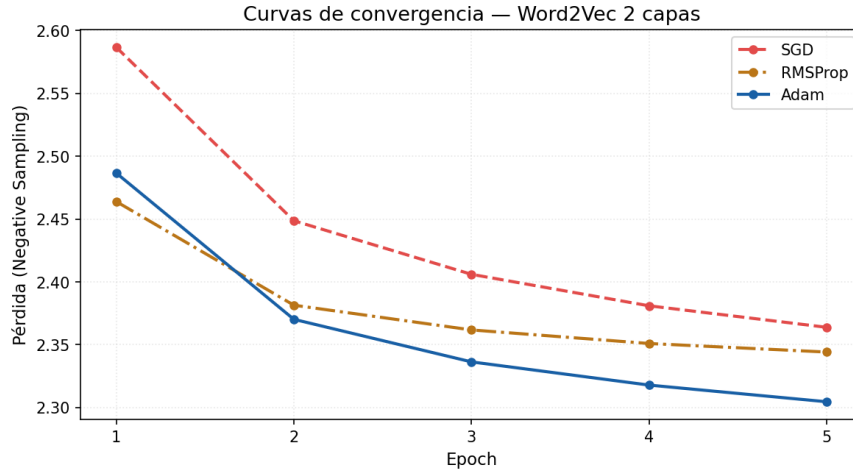


Figura 2: Curvas de convergencia — Word2Vec de 2 capas.

5.2. Resultados de clasificación

El Cuadro 3 es el resultado central: qué tan bien clasifica cada combinación, medido con F1-macro (un promedio justo entre acertar positivos y negativos). El ganador es 1 capa con RMSProp.

Tabla 3: Clasificación en test (4000 reseñas). Cada optimizador con su η adecuada.

Arquitectura	Optimizador	η	Accuracy	F1-macro	Tiempo (s)
1 capa	SGD	0.1	0.8608	0.8607	221
1 capa	RMSProp	0.001	0.8940	0.8940	183
1 capa	Adam	0.001	0.8932	0.8932	188
2 capas	SGD	0.1	0.8812	0.8812	302
2 capas	RMSProp	0.001	0.8822	0.8822	181
2 capas	Adam	0.001	0.8900	0.8900	198

Qué significa. Tres lecturas rápidas: los métodos adaptativos (RMSProp, Adam) van un pelo por delante; la segunda capa *no* ayuda (las de 2 capas no superan a la mejor de 1 capa); y SGD, con su tasa, queda competitivo aunque no gana.

5.3. Visualización: ¿el mapa de palabras existe?

Para ver si Word2Vec de verdad aprendió un “mapa de palabras”, usamos t-SNE (una técnica que aplanar un espacio de muchas dimensiones a un plano de 2, como aplanar un globo terráqueo en un mapa). En la Figura 3, palabras relacionadas caen juntas: señal de que el modelo capturó estructura semántica.

—qué tan “empinado” siente el terreno cada uno— época por época (Figura 5). El gradiente de SGD es unas **80 veces más pequeño** que el de Adam al inicio (luego la brecha baja a ~ 7 veces). La explicación cae de madura: Adam *normaliza* el paso por el historial, así que da zancadas útiles donde SGD da pasos ridículamente pequeños y se queda en la meseta del arranque.

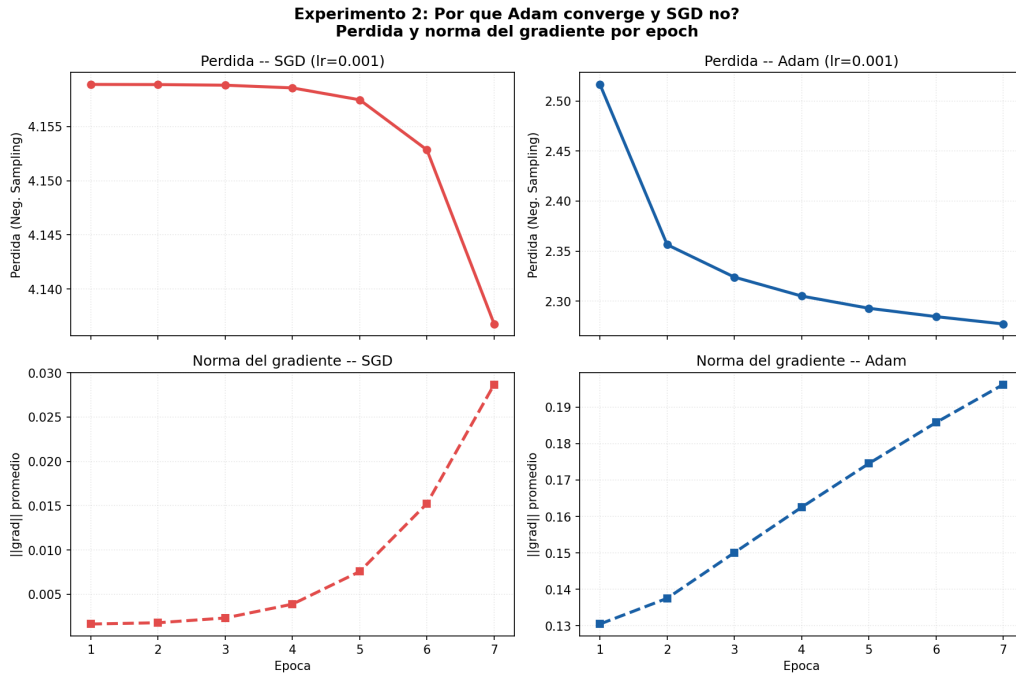


Figura 5: Pérdida y norma del gradiente para SGD y Adam, ambos con $\eta = 0,001$.

5.6. Experimento 3: el clásico TF-IDF entra al ring

TF-IDF es el método de la vieja escuela: básicamente cuenta qué palabras aparecen y qué tan distintivas son, sin entender nada de significado. Lo pusimos como rival. El resultado (Cuadro 5) nos sorprendió: TF-IDF gana, y lo hace *incluso en su versión más simple* (solo palabras sueltas, 0,9107), no solo con la versión más completa de pares de palabras (0,9225).

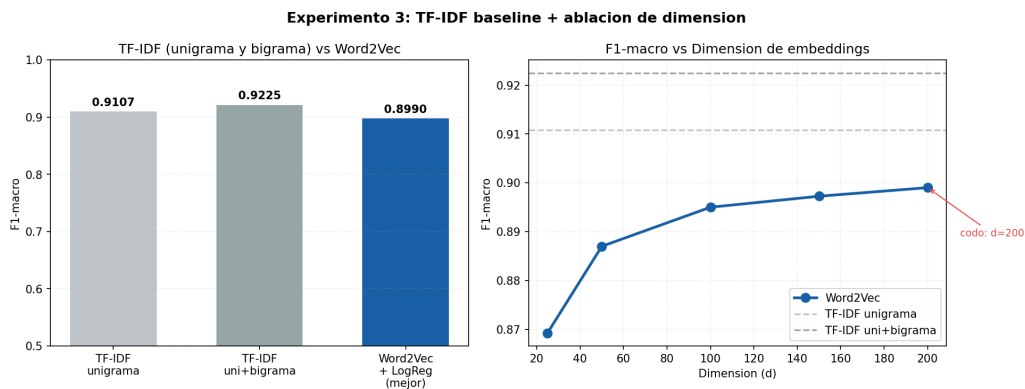


Figura 6: Izq.: TF-IDF (simple y con pares) vs. Word2Vec. Der.: F1 según la dimensión d .

Tabla 5: Dimensión de los embeddings (Adam) y el baseline TF-IDF.

Representación	F1-macro
Word2Vec $d = 25$	0.8692
Word2Vec $d = 50$	0.8870
Word2Vec $d = 100$	0.8950
Word2Vec $d = 150$	0.8972
Word2Vec $d = 200$	0.8990
TF-IDF palabras sueltas	0.9107
TF-IDF con pares de palabras	0.9225

Qué significa. Esto dice más del problema que de nuestro modelo. En las reseñas de Amazon la gente es directísima: “pésimo”, “no funciona”, “excelente”. Cuando la sola presencia de esas palabras ya delata el sentimiento, contarlas alcanza y sobra — trajimos un bisturí a una tarea que se resolvía con un martillo. Word2Vec rinde cuando el contexto importa más que la palabra suelta (ironía, dobles sentidos), algo que confirmaremos en el duelo final.

5.7. Experimento 4: ¿el modelo entendió de verdad?

Una cosa es clasificar bien y otra es que los vectores tengan sentido. Para verificarlo, le preguntamos al modelo cuáles son las palabras más parecidas a otras (Cuadro 6). Las respuestas son coherentes —incluye sinónimos y hasta faltas de ortografía del propio corpus—, lo que muestra que los vectores son interpretables, no una caja negra.

Tabla 6: Las 5 palabras más parecidas, por similitud coseno (Adam, $d = 100$).

Palabra	Vecinos más cercanos (similitud)
excelente	inmejorable (0.79), exelente (0.76), buenísimo (0.75), excepcional (0.74)
horrible	fatal (0.72), pésimo (0.66), malísimo (0.62), desastre (0.57)
roto	doblado (0.76), rajado (0.75), estropeado (0.73), destrozado (0.72)
recomiendo	aconsejo (0.77), recomendaría (0.70), recomendaria (0.69), volvería (0.64)
envío	envío (0.84), envió (0.80), entrega (0.70), estimado (0.69)

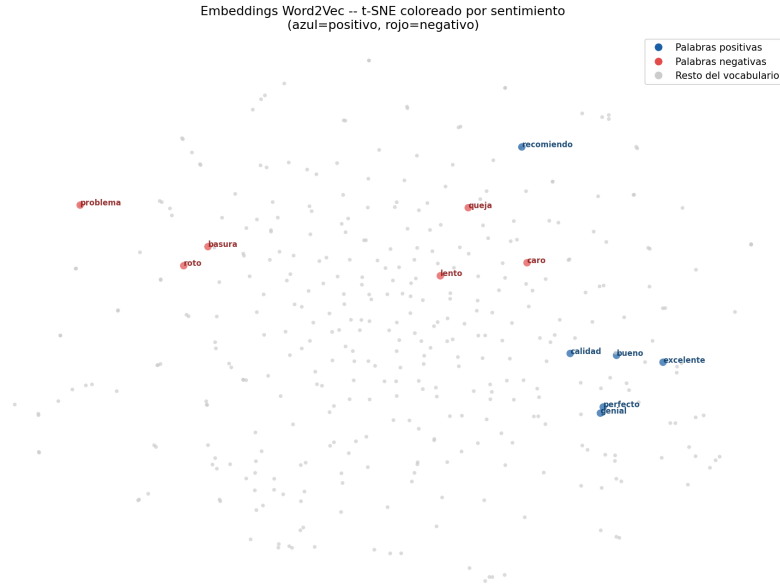


Figura 7: t-SNE coloreado: azul = palabras positivas, rojo = negativas.

5.8. Experimentos 5 y 5b: buscando los mejores ajustes

¿Cuáles son la tasa y la dimensión ideales? Lo buscamos de dos formas: una rejilla ordenada (probar combinaciones fijas) y una búsqueda Bayesiana con Optuna (que aprende de los intentos anteriores para proponer mejores). Las dos llegan prácticamente al mismo lugar (Cuadro 7). Importante: el óptimo de la rejilla quedó en el *centro* del rango probado (peor por encima y por debajo), señal de que buscamos en el lugar correcto.

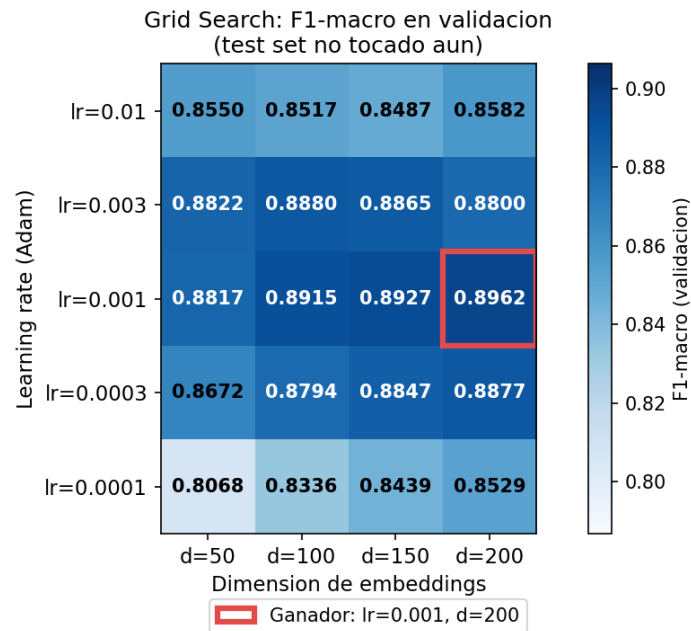


Figura 8: Mapa de calor de la rejilla. El óptimo ($\eta = 0,001$, $d = 200$) cae en el interior.

Tabla 7: Rejilla vs. búsqueda Bayesiana.

Método	Mejor config.	F1-val	F1-test
Rejilla	$\eta = 0,001$, $d = 200$	0.8962	0.8972
Optuna (TPE)	$\eta = 0,00059$, $d = 166$	0.8980	0.8970

Qué significa. La búsqueda Bayesiana, más sofisticada, no le ganó a la rejilla. No siempre lo complejo paga: aquí el paisaje de validación es plano alrededor del óptimo, así que una rejilla bien diseñada es más barata y igual de buena.

5.9. Experimento 6: el duelo contra el gigante

Este es el experimento que más nos hizo reflexionar. La pregunta era directa: nuestro Word2Vec, entrenado con 40 mil reseñas, ¿puede competir con SBWC, unos embeddings preentrenados con **1 500 millones** de palabras? Para que la comparación fuera justa, ambos usan el mismo clasificador y preprocesamiento; lo único que cambia es de dónde salen los vectores. Medimos en tres escenarios: Amazon (nuestro dominio), y tuits en español [7] de dos formas —transferencia (clasificador entrenado en Amazon y probado en tuits) e “in-domain” (clasificador entrenado sobre los propios tuits)— para separar el efecto del cambio de tema del efecto de la calidad del vector.

Tabla 8: El duelo (F1-macro). Mismo clasificador y preprocesamiento para todos.

Representación	Amazon (su dominio)	Tuits (transfer.)	Tuits (in-domain)
Nuestro W2V ($d = 300$)	0.9027	0.6733	0.7129
SBWC preentrenado	0.8830	0.7529	0.7589
TF-IDF (referencia)	0.9225	0.6829	0.7222

Qué significa. La respuesta intuitiva sería “el gigante gana de calle”, pero el resultado tiene un giro. En *nuestro* dominio —las reseñas de Amazon— nuestro modesto modelo le *gana* al gigante (0,903 vs. 0,883), porque conoce el vocabulario específico: “exelente” mal escrito, “no funciona”, nombres de productos. Pero al sacarlo de su zona y evaluarlo con tuits —un lenguaje informal, lleno de palabras que nunca vio— la situación se invierte y SBWC pasa al frente (0,759 vs. 0,713). La explicación cabe en un número: la *cobertura* de vocabulario (Figura 9). De cada 100 palabras de los tuits, nuestro modelo reconoce 82,7 y SBWC, 96,4. Las palabras que el modelo no conoce simplemente desaparecen del promedio y, con ellas, parte de la señal. La variante “in-domain” confirma que la diferencia no es del clasificador sino del vector mismo: aun entrenando sobre tuits, nuestro modelo choca con ese techo de vocabulario. La moraleja no es “el preentrenado es mejor”, sino algo más fino: **lo especializado rinde mejor en su dominio; lo general transfiere mejor fuera de él.**

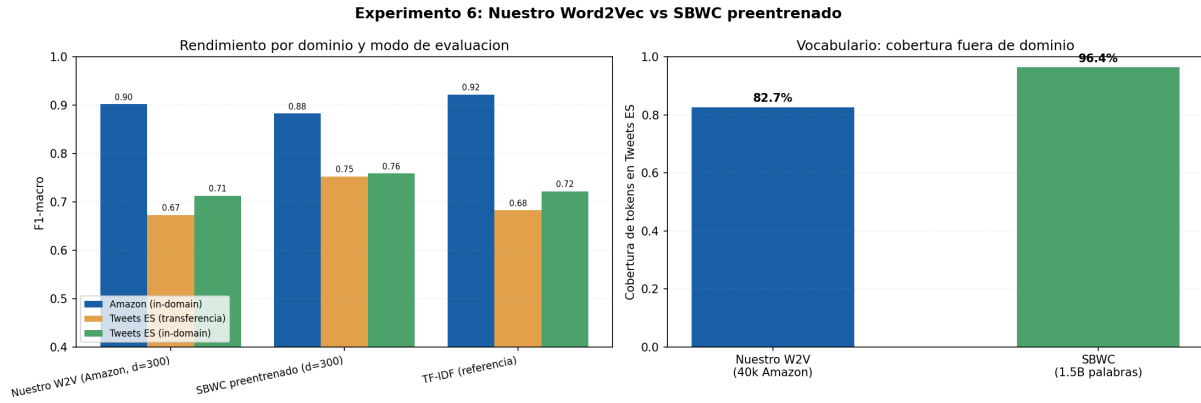


Figura 9: Duelo W2V vs. SBWC: rendimiento por dominio (izq.) y cobertura de vocabulario en los tuits (der.).

5.10. Verificación del gradiente

Para cerrar el círculo entre la teoría y la práctica, comprobamos que el gradiente que derivamos a mano (Ecuación (3)) es realmente correcto. Lo comparamos contra dos referencias: (A) la diferenciación automática de PyTorch y (B) un gradiente numérico calculado por diferencias finitas (mover un poquito cada parámetro y ver cuánto cambia la pérdida). Las tres coinciden: el error máximo entre nuestra fórmula y la automática es del orden de 10^{-16} , y la prueba de diferencias finitas pasa. En otras palabras, no solo escribimos la fórmula bonita: comprobamos que el modelo minimiza la pérdida exacta con su gradiente exacto.

6. Análisis y discusión

Volvamos a las hipótesis y veamos cuáles aguantaron.

H1 (confirmada, con matiz). A igual tasa ($\eta = 0,001$), los adaptativos le sacan mucha ventaja a SGD, que se queda atascado (exp2). Pero ojo: eso no significa que SGD sea peor. Con su propia tasa ($\eta = 0,1$) llega a 0,861, a solo ~ 3 puntos del mejor. La diferencia es de *ajuste*, no de talento del algoritmo.

H3 (confirmada). SGD es muy sensible a la tasa: barre de 0,62 a 0,87 según el paso (exp1), mientras Adam funciona bien en un rango amplio. El exp2 explica el mecanismo: la inicialización en ceros crea una meseta casi plana, y solo la inercia y la adaptación del paso permiten escapar de ella.

H2 (refutada, con matiz). La segunda capa no mejoró la clasificación: el mejor modelo sigue siendo 1 capa con RMSProp (0,8940). Curiosamente, esa capa extra *sí* ayuda al optimizador más débil (SGD sube de 0,861 a 0,881): la no linealidad cambia el terreno de una forma que le conviene a SGD, aunque no mueva la aguja del mejor modelo global.

H4 (refutada). El resultado más instructivo del trabajo. TF-IDF nos gana, y lo hace incluso con palabras sueltas (0,9107 vs. 0,8990), así que no es cuestión de pares de palabras. Refleja una propiedad del dominio: en reseñas de productos, las palabras clave por sí solas bastan. No es una derrota de Word2Vec, es honestidad sobre cuándo conviene cada herramienta.

H5 (refutada). La búsqueda Bayesiana no superó significativamente a la rejilla (0,8970 vs. 0,8972). El óptimo es interior y el paisaje, plano: una rejilla bien pensada fue más costo-efectiva.

H6 (confirmada, con el matiz más interesante). SBWC generaliza mejor fuera de dominio (0,759 vs. 0,713 en tuits), respaldado por la cobertura (96,4 % vs. 82,7 %). Pero dentro de dominio la cosa se invierte: nuestro modelo le gana en Amazon (0,903 vs. 0,883). No hay un ganador absoluto, hay un *trade-off de dominio*.

Interpretabilidad. El exp4 confirma que los vectores tienen significado real (excelente $\rightarrow$ inmejorable, roto $\rightarrow$ doblado). Poder “leer” lo que aprendió el modelo es una ventaja frente a métodos más opacos.

Sobre nuestras decisiones de diseño. Para que no parezcan azar: (i) cada optimizador usó su tasa adecuada, porque compararlos con la misma sería injusto; (ii) exp1 y exp2 usan 7 épocas en vez de 5 para darle tiempo a la dinámica lenta de SGD; (iii) los experimentos de afinación usan Adam, que está empatado con RMSProp ($< 0,001$) y es el estándar; (iv) el duelo usa $d = 300$ para igualar la dimensión de SBWC, dentro de la meseta de rendimiento.

7. Fortalezas, limitaciones y usos profesionales

7.1. Fortalezas

Eficiencia: nuestro Word2Vec representa cada reseña con 300 números y queda a 1–2 puntos de un TF-IDF de 50 000 características. Quedar tan cerca usando 167 veces menos números es, en realidad, una virtud. **Transferibilidad:** los vectores se reutilizan en otras tareas, como mostró el duelo fuera de dominio. **Interpretabilidad:** distancia entre vectores = similitud de significado (exp4). **Sin etiquetas:** se entrena solo con texto crudo.

7.2. Limitaciones

Vectores estáticos: “banco” (de plata) y “banco” (de sentarse) comparten un único vector. **Vocabulario cerrado:** las palabras nuevas no tienen representación —lo medimos en el exp6, donde nuestra cobertura fuera de dominio fue del 82,7 % frente al 96,4 % de SBWC. **Promedio del documento:** al promediar se pierde el orden de las palabras, y con él los matices que dependen de la estructura de la frase.

7.3. Usos actuales en el ámbito profesional

Word2Vec y sus variantes siguen muy vivos: en motores de recomendación (tratando productos como “palabras” en secuencias de compra), en búsqueda semántica y recuperación de información, en análisis de sentimiento y enrutamiento de tickets de atención al cliente, en expansión de consultas, y como punto de partida para modelos más grandes. Su bajo costo, su velocidad y su interpretabilidad lo mantienen como una herramienta de primera línea cuando importan la latencia y los recursos.

8. Conclusiones

1. **H1 confirmada:** a igual tasa, Adam y RMSProp convergen y SGD se atasca (gradiente $\sim 80\times$ menor); pero con su tasa, SGD llega a 0,861. El factor decisivo es el ajuste, no el algoritmo.
2. **H2 refutada:** la segunda capa no mejora la clasificación. El mejor modelo es 1 capa con RMSProp (F1= 0,8940).
3. **H3 confirmada:** SGD exige ajuste fino de la tasa (F1 de 0,62 a 0,87); Adam es robusto.

4. **H4 refutada:** TF-IDF supera a Word2Vec incluso con palabras sueltas (0,9107 vs. 0,8990); el vocabulario directo de las reseñas favorece al método clásico.
5. **H5 refutada:** la búsqueda Bayesiana no mejora a la rejilla; el óptimo es interior y el paisaje, plano.
6. **H6 confirmada con matiz:** SBWC generaliza mejor fuera de dominio, pero nuestro modelo lo supera en su propio dominio. Es un trade-off, no un ganador absoluto.
7. **Para el curso:** Word2Vec resultó un caso de estudio ideal de optimización — pudimos ver la función de pérdida, derivar y *verificar* su gradiente, comparar algoritmos de minimización y buscar hiperparámetros— todo sobre un problema real de NLP en español.

Referencias

- [1] Z. S. Harris. *Distributional structure*. Word, 10(2–3):146–162, 1954.
- [2] P. Keung, Y. Lu, G. Szarvas, N. A. Smith. *The Multilingual Amazon Reviews Corpus*. EMNLP, 2020.
- [3] D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980, 2014.
- [4] T. Mikolov, K. Chen, G. Corrado, J. Dean. *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781, 2013.
- [5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean. *Distributed Representations of Words and Phrases and their Compositionality*. NeurIPS, 2013.
- [6] N. Muennighoff, N. Tazi, L. Magne, N. Reimers. *MTEB: Massive Text Embedding Benchmark*. arXiv:2210.07316, 2023.
- [7] F. Barbieri, L. Espinosa Anke, J. Camacho-Collados. *XLM-T: Multilingual Language Models in Twitter for Sentiment Analysis and Beyond*. LREC, 2022.